第十二讲教学设计：Listener 监听器与Web应用生命周期管理 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: Listener监听器与Web应用生命周期管理
- **授课时长**: 20分钟
- **适用对象**: 已掌握Servlet/JSP、会话管理与过滤器基础，准备完善应用级治理能力的学生

一、教学目标

知识目标

1. 认识Servlet监听器（Listener）的分类与作用：
   - 生命周期监听器（Lifecycle Listeners）：`ServletContextListener`、`HttpSessionListener`、`ServletRequestListener`。
   - 属性监听器（Attribute Listeners）：`ServletContextAttributeListener`、`HttpSessionAttributeListener`、`ServletRequestAttributeListener`。
2. 掌握注解（Annotation）`@WebListener`与`web.xml`两种注册方式。
3. 理解典型场景：应用启动初始化（Initialization）、资源释放（Resource Cleanup）、在线人数统计（Online Count）、请求耗时追踪（Request Timing）、审计与风控（Audit）。

能力目标

1. 能实现“应用启动加载配置”“在线人数统计”“请求耗时追踪”三个实用监听器。
2. 能在监听器中正确使用应用作用域（`ServletContext`）存取跨请求共享的数据，并处理并发安全。
3. 能设计监听事件的最小化逻辑，避免阻塞主请求链路。

素质目标

1. 强化全局观：从“单个请求”走向“应用全生命周期”的工程治理意识。
2. 培养风险意识：启动与销毁阶段的资源管理与异常兜底。
3. **[思政融合]** 借“城市运行中心”的比喻，强调“未雨绸缪、统筹治理”的现代治理理念，保障系统有序高效与群众获得感。

二、教学重点与难点

教学重点

- 监听器触发时机与作用域：应用、会话、请求三级。
- 典型监听器的组合场景：启动初始化 + 在线人数统计 + 请求耗时追踪。
- 注解与`web.xml`注册差异与适用场景。

教学难点

- 并发安全（如在线人数计数器）：使用`AtomicInteger`或同步策略。

- 监听器中避免长耗时与阻塞，必要时异步落库/日志。
- 应用重启/热部署导致状态重建的应对（幂等初始化）。

三、教学内容与时间安排

第一部分：问题引入——谁来在“系统启动/关闭”时干活？（4分钟）

内容要点

1. 场景：应用需在启动时加载配置/字典数据，关闭时释放连接池/线程池。
2. 引出监听器：容器在关键生命周期节点回调我们的代码，实现“上电自检、断电保护”。

教学方法

- 场景驱动 + 类比法（城市早高峰调度/夜间保洁）。

第二部分：新知讲解——监听器体系与实现（9分钟）

内容要点

1. 应用启动初始化（`ServletContextListener`）：

```java
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebListener;
import java.io.InputStream;
import java.util.Properties;
import java.util.concurrent.atomic.AtomicInteger;

@WebListener
public class AppBootstrapListener implements ServletContextListener {
    @Override
    public void contextInitialized(ServletContextEvent sce) {
        ServletContext ctx = sce.getServletContext();
        // 1) 读取配置
        Properties props = new Properties();
        try (InputStream in = ctx.getResourceAsStream("/WEB-INF/app.properties")) {
            if (in != null) props.load(in);
        } catch (Exception e) { e.printStackTrace(); }
        ctx.setAttribute("appConfig", props);
        // 2) 初始化在线人数计数器
        ctx.setAttribute("onlineCount", new AtomicInteger(0));
    }
    @Override
    public void contextDestroyed(ServletContextEvent sce) {
        // 释放资源（如关闭连接池/定时任务），示意即可
    }
}
```

2. 在线人数统计（`HttpSessionListener`）：

```java
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebListener;
import jakarta.servlet.http.*;
import java.util.concurrent.atomic.AtomicInteger;

@WebListener
public class SessionCounterListener implements HttpSessionListener {
    @Override
    public void sessionCreated(HttpSessionEvent se) {
        ServletContext ctx = se.getSession().getServletContext();
        AtomicInteger online = (AtomicInteger) ctx.getAttribute("onlineCount");
        if (online != null) online.incrementAndGet();
    }
    @Override
    public void sessionDestroyed(HttpSessionEvent se) {
        ServletContext ctx = se.getSession().getServletContext();
        AtomicInteger online = (AtomicInteger) ctx.getAttribute("onlineCount");
        if (online != null) online.decrementAndGet();
    }
}
```

3. 请求耗时追踪（`ServletRequestListener`）：

```java
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebListener;

@WebListener
public class RequestTimingListener implements ServletRequestListener {
    @Override
    public void requestInitialized(ServletRequestEvent sre) {
        sre.getServletRequest().setAttribute("startedAt",
System.currentTimeMillis());
    }
    @Override
    public void requestDestroyed(ServletRequestEvent sre) {
        Object val = sre.getServletRequest().getAttribute("startedAt");
        if (val instanceof Long) {
            long cost = System.currentTimeMillis() - (Long) val;
            System.out.println("[TIMING] cost=" + cost + "ms");
        }
    }
}
```

4. 属性监听（可选审计，`HttpSessionAttributeListener`）：

```java
@WebListener
public class SessionAttrAuditListener implements HttpSessionAttributeListener {
    @Override
    public void attributeAdded(HttpSessionBindingEvent event) {
        System.out.println("ADD " + event.getName());
    }
    @Override
    public void attributeRemoved(HttpSessionBindingEvent event) {
        System.out.println("REMOVE " + event.getName());
    }
    @Override
    public void attributeReplaced(HttpSessionBindingEvent event) {
        System.out.println("REPLACE " + event.getName());
    }
}
```

5. `web.xml` 注册方式（可替代注解）：

```xml
<listener>
    <listener-class>com.demo.listener.AppBootstrapListener</listener-class>
</listener>
<listener>
    <listener-class>com.demo.listener.SessionCounterListener</listener-class>
</listener>
<listener>
    <listener-class>com.demo.listener.RequestTimingListener</listener-class>
</listener>
```

教学方法

- 代码走查 + 触发时机演示（启动/新建Session/请求开始与结束）。

第三部分：实战——显示在线人数与读取系统参数 (5分钟)

内容要点

1. 在JSP页展示在线人数：

```
当前在线人数：${applicationScope.onlineCount}
```

2. 从应用作用域读取配置：

```
系统名称：${applicationScope.appConfig['app.name']}
```

3. 验证：新开/关闭浏览器窗口，观察在线人数的增减；请求控制台打印耗时。

教学方法

- 现场联调 + 多浏览器对比实验。

第四部分：课堂总结与展望 (2分钟)

总结要点

1. 监听器提供容器级生命周期与属性变更的“观察点”。
2. 常见组合：启动初始化 + 在线统计 + 请求耗时追踪 + 属性审计。
3. 注意并发安全与轻量化处理，重活可异步化。

展望（承上启下）

- 下一讲将进入“三层架构与DAO模式重构”，把数据访问从业务中抽离，进一步提升可维护性与可测试性。

四、教学方法与手段

教学方法

1. **场景驱动 (Scenario-Driven)**: 从应用启动/关闭需求入手
2. **代码走查 (Code Walkthrough)**: 讲解各类Listener实现
3. **实验验证 (Experiment)**: 多浏览器验证在线人数统计
4. **类比法 (Analogy)**: "城市运行中心"比喻
5. **双语教学法 (Bilingual Teaching)**:
    - 课堂采用英文授课，关键技术术语提供中英文对照
    - 重要概念（Listener, Lifecycle, Initialization, Resource Cleanup, Online Count）首次出现时给出英文表达和中文释义
    - 鼓励学生使用英文术语描述监听器机制
    - 代码注释和变量命名使用规范英文

教学手段

- IDE (IntelliJ IDEA/Eclipse)
- Tomcat服务器 (Tomcat Server)
- 浏览器开发者工具 (Browser DevTools)
- PPT/Marp课件 (Presentation Slides)

五、教学评价

- **过程性评价**: 能否说出三类生命周期监听器的触发时机；能否解释在线人数计数的并发安全做法。
- **终结性评价**: 能否实现并演示启动初始化、在线统计、请求耗时追踪三个监听器。

六、课后拓展

思考题

1. 如果应用热部署（redeploy），在线人数如何校准？有哪些可行的、成本较低的策略？
2. 在 `RequestTimingListener` 中直接写文件日志可能造成什么问题？如何改造以降低对请求的影响？

实践作业

1. 增加 `HttpSessionAttributeListener` 实现登录用户审计：当 `loggedInAgent` 被设置/清理时记录日志。

2. 将“请求耗时”改造为异步落库（或缓冲批量写入），并添加超过阈值的告警统计。